C Libraries in Swift, Explained for Windows

A C library always has two parts: the header (`.h`), which says what functions exist, and the compiled code, which actually does the work. You always need both. The only real choice is *who compiles that code*. With SQLite's amalgamation, you get the source and Swift compiles it as part of your project. In the Hacking with Swift tutorial, someone else already compiled GD and you only point Swift at the header plus the finished binary. Below you'll build one of each on Windows, so you can see the difference for yourself.

I. The mental model

1. The files you'll see

Think of a restaurant. The header is the menu. The source is the recipe. The compiled library is the finished dish.

FILE	WHAT IT CONTAINS	WHO USES IT
`.h` header	Function *declarations* only: names, parameters, types. No working code.	The Swift compiler, so it knows what it's allowed to call
`.c` source	The actual implementation	A C compiler, which turns it into machine code
`.lib` (Windows)	Either the full compiled code (a *static* library), or a small "import library" that points to a DLL	The linker, at build time
`.dll` (Windows)	The compiled code, loaded when your program starts	Windows, at run time

The Mac and Linux equivalents: `.dylib` / `.so` instead of `.dll`, and `.a` instead of a static `.lib`.

2. Why the tutorial "only uses the .h"

It doesn't use only the header. The compiled code just comes from somewhere you don't see. When the page runs `brew install gd`, Homebrew downloads GD already compiled. The module map then has two lines that do two different jobs:

```
header "/usr/local/include/gd.h"   // the menu:
link "gd"                          // the dish:
```

Delete the `link` line and Swift still recognizes `gdImageFill`, because it read the header. But the build fails at the very end, because there's no code behind that function.

3. The two ways to get the code

	A. BUILD FROM SOURCE	B. USE A PREBUILT LIBRARY
You put in your project	`.h` + `.c`	A module map that points to an installed `.h` + `.lib`
Who compiles the C code	Swift Package Manager (SPM), every time you build	Someone else, ahead of time (brew, apt, vcpkg)
Install step on each machine	None	Yes
Paths and DLLs to manage	None	Include path, lib path, DLLs at runtime
Works when	The library is small with no dependencies	The library is big or depends on other libraries
Example	SQLite	GD (the page), OpenSSL, libxml2

4. Why SQLite and GD get different advice

SQLite is designed to be built from source. GD isn't.

- The SQLite "amalgamation" is the whole database engine merged into one file, `sqlite3.c`, plus `sqlite3.h`. It has no dependencies, and SQLite's own docs recommend using it.
- Building it yourself means every developer and every server gets the same SQLite version, and you can switch on optional features like full-text search with a compiler flag.

- GD is dozens of source files, and it needs libpng, zlib, libjpeg and freetype. Building it yourself means building all of those too. On Mac and Linux it's one install command, so the page uses the prebuilt version.

5. About the "shim.h" from earlier

It isn't source code. It's a one-line header that says "go find gd.h". The page hardcodes `/usr/local/include/gd.h`, and that's the only reason it needs separate Mac and Linux modules. A shim containing `#include <gd.h>` lets the compiler look up `gd.h` in whatever include folders you give it, so one module map works on every OS.

II. Tutorial A: SQLite from source (nothing to install)

Step 1. Get the amalgamation

From sqlite.org's Download page, get sqlite-amalgamation-*.zip (the first "Source Code" entry) and unzip it. You only need `sqlite3.c` and `sqlite3.h`.

Create this layout. In PowerShell, `mkdir` creates the in-between folders for you:

```
mkdir SQLiteDemo\Sources\CSQLite\include
mkdir SQLiteDemo\Sources\SQLiteDemo
cd SQLiteDemo
```

```
SQLiteDemo\
  Package.swift
  Sources\
    CSQLite\
      sqlite3.c              <- the recipe (sou
      include\sqlite3.h      <- the menu (heade
    SQLiteDemo\
      main.swift
```

The `include` folder matters. SPM treats any header in it as public and generates the module map for you, so you don't write one.

Step 2. Package.swift

```
// swift-tools-version:6.0
import PackageDescription

let package = Package(
    name: "SQLiteDemo",
    targets: [
        // A normal C target: SPM compiles sqli
        .target(
            name: "CSQLite",
            cSettings: [.define("SQLITE_ENABLE_
        ),
        .executableTarget(name: "SQLiteDemo", de
    ]
)
```

Compare this with the page: there's no `.systemLibrary`, no `link` line and no paths, because the compiled code comes from your own build.

Step 3. Call raw C from Swift

`Sources\SQLiteDemo\main.swift`

```
import CSQLite

var db: OpaquePointer?                        //
guard sqlite3_open("demo.db", &db) == SQLITE_OK
defer { sqlite3_close(db) }                   //

sqlite3_exec(db, "CREATE TABLE IF NOT EXISTS pe
sqlite3_exec(db, "INSERT INTO people VALUES ('Ar

var stmt: OpaquePointer?
sqlite3_prepare_v2(db, "SELECT name, age FROM pe
defer { sqlite3_finalize(stmt) }

while sqlite3_step(stmt) == SQLITE_ROW {
    let name = String(cString: sqlite3_column_te
    let age  = sqlite3_column_int(stmt, 1)
    print(name, age)
}
```

```
swift run
```

The first build takes a little longer because it compiles about 250,000 lines of C. After that, builds are cached. This is the equivalent of the page's "Testing the wrapper" step: it works, but it reads like C.

Step 4. Wrap the wrapper

Same idea as the page's `Image` class: use a class so that `deinit` frees the C resource automatically. Add this to `main.swift`, or put it in its own library target:

```
final class Database {
    private var handle: OpaquePointer?

    init(path: String) throws {
        guard sqlite3_open(path, &handle) == SQL
    }
    deinit { sqlite3_close(handle) }          //

    func execute(_ sql: String) {
        sqlite3_exec(handle, sql, nil, nil, nil)
    }

    func rows(_ sql: String) -> [[String]] {
        var stmt: OpaquePointer?
        sqlite3_prepare_v2(handle, sql, -1, &str
        defer { sqlite3_finalize(stmt) }
        var result: [[String]] = []
        while sqlite3_step(stmt) == SQLITE_ROW {
            let count = sqlite3_column_count(str
            result.append((0..<count).map { Str
        }
        return result
    }
}
enum DBError: Error { case open }

let people = try Database(path: "demo.db")
print(people.rows("SELECT name, age FROM people
```

This is a learning example. Real code would check every return code, and would use `sqlite3_bind_*` instead of building SQL by joining strings.

III. Tutorial B: GD prebuilt (the page's approach, on Windows)

Step 1. Get GD already compiled

vcpkg is Microsoft's C/C++ package manager. On Windows it does the job brew and apt do on the page.

```
git clone https://github.com/microsoft/vcpkg C:\
C:\vcpkg\bootstrap-vcpkg.bat
C:\vcpkg\vcpkg install libgd:x64-windows
```

Now look at what you got. These are the three pieces from Section I:

```
C:\vcpkg\installed\x64-windows\include\gd.h
C:\vcpkg\installed\x64-windows\lib\libgd.lib
C:\vcpkg\installed\x64-windows\bin\libgd.dll
                                        (+ libpng16
```

Check the actual `.lib` name in that folder. It's usually `libgd.lib` on Windows. Use that name without the `.lib` extension in the next step.

Step 2. Describe it to Swift

```
GDDemo\
  Package.swift
  Sources\
    Cgd\module.modulemap
    Cgd\shim.h
    GDDemo\main.swift
```

`Package.swift`

```
// swift-tools-version:6.0
import PackageDescription

let package = Package(
    name: "GDDemo",
    targets: [
        .systemLibrary(name: "Cgd"),        // '
        .executableTarget(name: "GDDemo", depend
    ]
)
```

`Sources\Cgd\shim.h`

```
#include <gd.h>
```

`Sources\Cgd\module.modulemap`

```
module Cgd [system] {
    header "shim.h"
    link "libgd"      // must match libgd.lib fr
    export *
}
```

Step 3. The page's code, Windows edition

`Sources\GDDemo\main.swift`

```
import ucrt       // Windows' C runtime, used ir
import Cgd

let image = gdImageCreateTrueColor(800, 600)
defer { gdImageDestroy(image) }

let red  = gdImageColorAllocateAlpha(image, 255,
let blue = gdImageColorAllocateAlpha(image, 0, 0
gdImageFill(image, 0, 0, red)
gdImageFilledEllipse(image, 400, 300, 200, 100,

let file = fopen("output.png", "wb")   // "b" ma
defer { fclose(file) }
gdImagePng(image, file)
```

Unlike Tutorial A, you must tell the build where the header and the library are, and tell Windows where the DLLs are:

```
$env:PATH += ";C:\vcpkg\installed\x64-windows\bi
swift run -Xcc -IC:\vcpkg\installed\x64-windows\
```

`-Xcc -I` says where the menu is. `-Xswiftc -L` says where the linker should look for `libgd.lib`. `PATH` says where Windows should look for the DLLs when the program starts. Those are three separate settings because they serve the three pieces from Section I. Swift may show a deprecation warning for `fopen` on Windows. It's harmless.

From here, the page's `Color` / `Image` wrapper works as written. Just use the `ucrt` import above instead of the `Darwin`/`Glibc` block.

IV. Reading the errors

Each error points to one missing piece

Once you know the three pieces, Windows build errors stop being mysterious.

YOU SEE	WHEN	MISSING PIECE	FIX
`'gd.h' file not found` or `no such module 'Cgd'`	Compiling	The header	Fix `-Xcc -I…`, or check folder names match the target names
`LNK2019: unresolved external symbol gdImageFill`	Linking	The compiled code (`.lib`)	Fix `-Xswiftc -L…` or the name in `link "…"`
"libgd.dll was not found", or the program exits silently	Running	The DLL	Add vcpkg's `bin` to `PATH`, or copy the DLLs next to the `.exe`

Tutorial A can only hit the first error. The C code is compiled straight into your `.exe`, so there's no separate library to link and no DLL to find.

Rule of thumb for work

- Small library with no dependencies (SQLite, cJSON, stb, miniz): add the source as a C target. On Windows this is usually the easiest option.
- Big library, or one with dependencies (GD, OpenSSL, libcurl, ICU): use a prebuilt library from vcpkg with `.systemLibrary`, and plan how the DLLs will ship with your app.
- Either way, write a small Swift class on top whose `deinit` frees the C resource. That's the main lesson of the page, and it applies the same way on Windows.

Made with Dia